CloudWatch Log Insights

Explore how to use CloudWatch Logs Insights to interactively query and visualize log data across AWS services. Learn to write queries for filtering, aggregating, and sorting logs to troubleshoot issues and monitor application performance efficiently.

As the application grows in complexity, so does its volume of logs. Reading logs line-by-line quickly becomes overwhelming, especially when errors are intermittent or spread across multiple services.

CloudWatch Logs Insights is a powerful, interactive log analytics tool that allows us to search, filter, and visualize log data across AWS resources in real time. With a few lines of query syntax, we can uncover patterns, troubleshoot issues, and monitor behavior.

A developer team is investigating a latency issue in their serverless checkout workflow. They open the CloudWatch Logs console and click into different Lambda log streams. Within minutes, they're drowning in hundreds of log lines, none related to the delay.

Ask

That's where CloudWatch Logs Insights makes a difference. Logs Insights gives us a query interface, like SQL for logs. Instead of scrolling endlessly, we can ask questions and instantly see only the relevant data.

How does CloudWatch Logs Insights work?

Logs Insights works on log groups containing log streams emitted by AWS services like Lambda, EC2, ECS, and API Gateway. Once the logs are in CloudWatch, we can select a group, choose a time range, and run structured queries to:

- Filter logs by keywords, fields, or conditions.
- Extract JSON fields from structured logs.
- Aggregate data over time (e.g., error counts every 5 minutes).
- Sort, limit, and visualize results.

This feature does not require any pre-setup. If the application logs into CloudWatch, we can start querying immediately. Let's break down some of the core query operations supported in Logs Insights:

- `fields`: We can specify which fields from our logs we want to see. For example, `fields @timestamp, @message` will display the timestamp and the message content of each log entry.
- `filter`: We can also narrow down our logs to only those that meet certain criteria. An example like `filter @message like /ERROR/` will show us only log entries containing the word "ERROR."
- `stats`: We can also perform aggregations on our log data, such as counting occurrences, calculating averages, or finding minimum and maximum values. For instance, `stats count(*) by bin(5m)` will count all log entries in 5 minute intervals.
- `parse`: The `parse` command helps us extract specific values from our log messages. For example, `parse @message '"user":"*"' as user` extracts

Ask

the value associated with `"user"` from a JSON-formatted log message and names it `user`.

- `sort` and `limit`: This command sequence is used in CloudWatch Logs Insights to retrieve a specific set of log entries. We can start with `sort @timestamp desc` to arrange our log entries from the newest (most recent) to the oldest. So, we'll see the latest logs first. We can then use `limit 10` to restrict the number of results we get back.

Advanced queries

We can combine the basic queries to perform more advanced queries. For example, we can combine filters and stats for insightful patterns. We often want to see how different regions or order types affect performance.

- **Analyzing performance and failures:** To calculate the **average latency**, excluding orders with zero latency, and group the results by `orderType` and `region`, we'd use this query:

main.sql MySQL

```
filter RequestLatency > 0
| stats avg(RequestLatency) by orderType, region
```

Ask

This query first **filters** out any log entries where `RequestLatency` is 0, then calculates the **average** `RequestLatency` for the remaining entries, **grouped** by both `orderType` and `region`.

- **Monitoring failures:** If we're only interested in monitoring failures, we can count them by region using this query:

main.sql C++

```
filter FailureCount > 0
| stats count(*) by region
```

This query **filters** for log entries where `FailureCount` is greater than 0, and then **counts** these occurrences, **grouped** by `region`.

This method transforms raw log data into actionable information. To write more efficient queries as our applications and log volumes grow, we should:

- **Filter early:** Apply filters as soon as possible in the query to reduce the dataset we're working with.
- **Use limit for debugging:** When interactively debugging, use the `limit` command to restrict the number of results, making the output more manageable.

Ask

- **Set a narrow time range:** Always specify a small time range to help reduce query costs and speed up results.
- **Structure logs:** Design our logs to be structured (e.g., in JSON format) to avoid needing complex regular expressions for parsing.

Visualizing query results

After running a query, Logs Insights allows us to visualize the results. We can switch between tables, bar charts, and line graphs. This is especially useful when we:

- Track metrics like latency or error count over time
- Compare categories, for example, Premium vs. Standard orders
- Validate business rules through log behavior. For example, a business might have a policy of triggering an additional fraud check on orders over $5000. We can run a query in Logs Insights to count both the qualifying orders and the fraud check events over time.

For example, to visualize error frequency during load testing, we can use the following query to measure the total number of failures per minute:

main.sql C++

```
filter FailureCount > 0
| stats sum(FailureCount) as totalFailures by bin(1m)
```

Ask

The query:

- Calculates the true volume of failures by summing the `FailureCount` field, giving us a better measure of the impact.
- Uses `as totalFailures` to rename the output column, making our graph's legend easier to understand.

We can then choose the "Line chart" view to see how error volume changes over time.

Integrating with CloudWatch dashboards

One of CloudWatch Logs Insights' most powerful features is adding the query results directly to a CloudWatch Dashboard. This allows us to create a unified observability dashboard with a complete story about the application's health.

Instead of switching between screens, we can place widgets side-by-side to correlate events at a glance. Imagine a dashboard for an e-commerce application that combines:

- **Metric graphs:** A line chart showing the average API latency (`p99`).
- **Alarm status:** An indicator showing the current state of our "High Latency" alarm.
- **Logs Insights widget:** A table generated by a query that shows the top 10 most common error messages from our application logs.

Now, when our latency alarm turns red, we don't have to go digging for the cause. We can look at the log widget right next to it and instantly see if a specific error, like "Database Timeout," spikes simultaneously. This transforms the dashboard from a simple monitoring tool into a powerful, interactive diagnostic surface.

Ask

Conclusion

CloudWatch Logs Insights takes what would otherwise be an overwhelming sea of log data and turns it into an on-demand analytics engine. When paired with structured logging, EMF, and custom metrics, it gives us real-time visibility and retrospective analysis in one tool.

By learning to write effective queries, we stop guessing and start understanding what is happening in our application. This capability is central to building

Ask